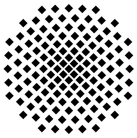


Universität
Stuttgart

Computational Cognitive Science

Replication and Pipeline Comparison of Mobile EEG Analyses in ds004033: Auditory Oddball, Walking Synchronization, and Decoding

Team Members:

Kishore Khan Shate Mohamed

Eashan Sai Urjann Chinni Krishnan

Georgina Shirazi

Tutor: Prof. Dr. Benedikt Ehinger

Supervisors: Christina Bohnacker, Maanik Yashodhan Marathe

Abstract

This report presents a reproduction and comparison study based on the OpenNeuro mobile EEG dataset ds004033, which combines an auditory oddball task and a walking synchronization task recorded from 18 participants across two sessions with active and passive electrode configurations. The project had three aims: to reconstruct an authors-inspired analysis pipeline as faithfully as practical in Python, to develop a more explicit and noise-robust preprocessing pipeline, and to compare both pipelines across shared downstream analyses. The workflow began with dataset audit and event normalization, followed by preprocessing, oddball ERP analysis, walking synchronization time-frequency analysis, and a time-resolved decoding extension. In the oddball branch, finalized event mapping enabled condition-specific ERP estimation for standing, walking alone, and walking together, together with deviant-minus-standard difference waves and peak-based summaries. In the synchronization branch, stride-aligned Morlet time-frequency analysis was used to extract alpha-mu and beta activity from predefined sensor regions of interest. The comparison showed that both pipelines supported the main analysis branches and produced broadly comparable group-level outputs, while the improved pipeline provided clearer configuration, validation, and quality-control handling. The decoding extension was completed for three pairwise condition contrasts, but mean window-level AUC remained near chance and is therefore interpreted cautiously as an exploratory analysis rather than a central result. Overall, the project demonstrates a reproducible and modular framework for reproducing and extending mobile EEG analyses in ds004033 while making methodological differences between preprocessing strategies explicit.

Keywords: mobile EEG, auditory oddball, walking synchronization, preprocessing comparison

Contents

Chapter 1

Introduction

1.1 Dataset and scientific background

EEG is often used to study fast brain responses linked to attention, perception, and action. In many standard EEG studies, the recordings are taken in controlled lab settings where movement is limited. Mobile EEG is more difficult because the data contain much more noise from motion, electrode changes, and other non-neural sources. Because of this, it is not enough to only clean the data well. The more important question is whether the final analysis still gives meaningful and believable results.

This report focuses on the OpenNeuro dataset `ds004033`. This dataset is useful because it combines two different EEG task types in one dataset: an auditory oddball task and a walking synchronization task [1]. It includes recordings from 18 participants across two sessions with active and passive electrode configurations. That makes it a good dataset for checking both standard ERP-style analyses and more movement-related EEG analyses.

The oddball task supports event-related potential (ERP) analysis, where responses are time-locked to sounds and averaged across trials. The walking synchronization task is different. It needs time-frequency analysis linked to the gait cycle, so it is more focused on ongoing oscillatory changes during movement. Since both tasks are available in the same dataset, this project can test whether one analysis framework can handle both in a consistent way.

1.2 Original study context and why `ds004033` is interesting

The dataset article makes it clear that `ds004033` contains data from two separate projects: the auditory oddball task and the walking synchronization task [1]. This is important because it means the dataset is not just useful for one single result. It gives two different analysis directions from the start. The walking synchronization branch is closer to the original movement-related study context, while the oddball branch gives a simpler and more familiar ERP-based analysis setting [2].

This makes the dataset especially useful for a semester project. It supports replication, comparison, and extension at the same time. The same subjects, sessions, and recordings can be processed with different pipelines and then compared across several downstream analyses. So the value of the dataset is not only in reproducing one figure. It is also in showing how choices in event mapping, preprocessing, and analysis affect the final results.

1.3 Project aim: authors-inspired replication, improved pipeline, and comparison

The main aim of this project was to build a reproducible Python-based framework for analysing `ds004033` in two parallel ways:

- an authors-inspired pipeline, built to follow the published analysis logic as closely as practical,
- our own improved pipeline, designed to be cleaner, more explicit, and easier to validate,
- and a direct comparison between the two.

The project followed one shared workflow. First, the dataset was audited and the event structure was checked. Next, the event labels were cleaned and mapped into the final analysis conditions. After

that, both preprocessing pipelines were run. The cleaned data were then used in three downstream branches:

- oddball ERP analysis,
- walking synchronization time-frequency analysis,
- and a decoding extension.

An important point is that this report is not only about preprocessing quality. The final goal is to see whether the analyses are scientifically meaningful and consistent. For that reason, the walking synchronization branch is treated as the main paper-style result, the oddball branch is treated as a strong complementary replication analysis, and decoding is presented more carefully as an exploratory extension.

1.4 Scope of the report

This report stays close to what was actually implemented in the final repository. It covers:

- the dataset and task structure,
- repository design and shared workflow,
- preprocessing in both pipelines,
- oddball ERP analysis,
- walking synchronization time-frequency analysis,
- decoding,
- and authors-versus-ours comparison results.

At the same time, the report does not claim more than the project really achieved. The authors-inspired branch is described as a reconstruction in Python/MNE [3], not as an exact copy of the original analysis environment. The decoding branch is included because it was fully implemented, but it is not treated as the main contribution of the project.

The rest of the report is organised as follows. Chapter 2 describes the dataset and the two main task branches. Chapter 3 explains the overall project strategy and repository design. Chapter 4 describes the two preprocessing pipelines and downstream methods. Chapter 5 presents the final results, and Chapter 6 discusses what was reproduced, what changed across pipelines, and what the limits of the project are. The report ends with the conclusion and appendices.

Chapter 2

Dataset and Experimental Tasks

2.1 Overview of `ds004033`

This project uses the OpenNeuro dataset `ds004033`. The main reason this dataset was suitable for the project is that it contains two different EEG task branches in the same dataset: an auditory oddball task and a walking synchronization task. This made it possible to compare the same two pipelines across both ERP-based and movement-related analyses.

The dataset also follows the BIDS format, which made the data structure and metadata easier to inspect and helped us build a shared workflow for preprocessing, analysis, and comparison.

2.2 Participants, sessions, and EEG configurations

The dataset contains recordings from 18 participants. Each participant completed two sessions using two EEG configurations: one active and one passive. This was relevant for the project because it added realistic variation in signal quality and artifact level.

Rather than collapsing everything too early, both sessions were kept throughout the workflow and later compared at the group level. This made the analysis more realistic and helped explain some of the early issues in the project, such as noisy channels and weak ERP structure.

2.3 Auditory oddball task

The first main task branch is the auditory oddball task. In the final workflow, the oddball events were mapped into three behavioral conditions: `standing`, `walking_alone`, and `walking_together`. Within each condition, standard and deviant events were analysed separately.

This branch was used for ERP analysis and produced condition-specific evoked responses, deviant-minus-standard difference waves, and peak-based summaries. It was also useful as a practical check on the preprocessing, since ERP waveforms quickly show whether event handling and artifact cleaning are working properly.

2.4 Walking synchronization task

The second main task branch is the walking synchronization task. This branch is closer to the movement-focused part of the dataset and to the original study direction.

Unlike the oddball branch, this task is based on gait-locked EEG activity over the stride cycle. In the final workflow, it used gait-event markers, stride alignment, Morlet time-frequency analysis, and ROI summaries in the alpha-mu and beta ranges. This branch was more challenging because it depended strongly on correct stride alignment and was more affected by movement-related noise.

2.5 Why the dataset supports two analysis tracks

A strong point of `ds004033` is that it supports two different analysis tracks within the same dataset. The oddball branch gave us ERP-based condition comparisons, while the synchronization branch gave us stride-locked time-frequency analysis. Together, these two branches gave a stronger basis for pipeline comparison than either one alone.

Chapter 3

Project Strategy and Repository Design

3.1 Reproduction philosophy: not exact duplication, but robustness-oriented replication

From the start, this project was not treated as an exact copy of the original authors' full analysis setup. That would have been difficult because our final implementation was built in Python and MNE, while the original work was based on a different tool environment. So instead of claiming exact duplication, we aimed for something more realistic: to follow the main analysis logic as closely as possible, while also building a second pipeline that was cleaner and easier to validate.

This led to the two-pipeline design used throughout the project:

- an **authors-inspired pipeline**, which tried to stay close to the published direction,
- and **our improved pipeline**, which used more explicit preprocessing, cleaner configuration, and clearer validation steps.

This was useful for two reasons. First, it made the comparison fairer. Instead of only building a new pipeline and saying it worked better, we had a reference branch to compare against. Second, it made the report stronger. The project was not only about cleaning EEG data, but about seeing whether different preprocessing choices changed the final scientific outputs.

Another important point is that this report is based on the final implemented workflow, not only on early project ideas. By the end of the project, the main analysis branches were completed and validated. Because of that, the authors-inspired branch can be described honestly as a reconstruction, our branch as a more explicit alternative, and the final comparison as a central part of the project.

3.2 Shared workflow: audit, mapping, preprocessing, analysis, and comparison

Even though the project had two pipelines, both were built on the same shared workflow. This was important because the later comparison would only make sense if both branches started from the same data and the same event definitions.

The full workflow followed these main stages:

- **Dataset audit:** checking the BIDS structure, available runs, and event files.
- **Event mapping:** inspecting and normalizing raw event labels into final analysis conditions.
- **Preprocessing:** running the authors-inspired and improved preprocessing branches in parallel.
- **Downstream analysis:** applying both pipelines to oddball ERP analysis, walking synchronization time-frequency analysis, and later decoding.
- **Comparison:** bringing both branches back together and comparing their outputs side by side.

This shared structure made the project easier to manage. Instead of having two separate codebases doing different things, both branches followed the same logic and only differed where the actual preprocessing choices changed. That made the later comparisons much more meaningful.

3.3 Repository organization and reproducibility choices

The final cleaned repository was organized to be easy to understand, rerun, and present. At the top level, it is mainly divided into:
`shared/`, `authors_pipeline/`, `ours_pipeline/`, `configs/`, `results/`

There is also a `data/` folder, but in the lightweight final version this mainly points to the dataset location rather than storing the full raw dataset.

The `shared/` folder contains the common project code. This includes the main notebooks, scripts, and reusable modules. The shared notebooks cover the whole workflow from audit to final comparison:

- `00_data_audit.ipynb`
- `01_event_mapping.ipynb`
- `02_preprocessing.ipynb`
- `03_oddball_replication.ipynb`
- `04_pipeline_comparison.ipynb`
- `05_sync_tfr_analysis.ipynb`
- `06_sync_comparison.ipynb`
- `07_decoding_analysis.ipynb`
- `08_decoding_comparison.ipynb`

The `authors_pipeline/` and `ours_pipeline/` folders contain the two main branches of the project. Their purpose is not to duplicate everything, but to keep outputs and pipeline-specific steps separated in a clean way.

The `configs/` folder stores the main YAML configuration files. This was one of the most useful design choices in the project, because it made the settings easier to inspect and reduced the chance of hidden parameter changes inside notebooks.

The `results/` folder stores the lightweight figures kept for reporting. Instead of including every large intermediate file, the final repo keeps selected outputs from preprocessing, ERP analysis, synchronization analysis, decoding, and the comparison stages. This made the final submission much easier to browse.

A final reproducibility choice was to keep both notebooks and scripts. The notebooks are helpful for understanding the workflow step by step, while the scripts are better for rerunning the full analysis more directly. Together, this gave the project a much cleaner and more reproducible structure than a loose set of experimental notebooks. A cleaned version of the final project repository, including the code, notebooks, configuration files, and selected figures used in this report, is available at GitHub repository.

Chapter 4

Methods

4.1 Event definitions and condition mapping

Both pipelines used the same cleaned event definitions. For the oddball branch, the final condition mapping was `standard1 / deviant1` for `standing`, `standard2 / deviant2` for `walking_alone`, and `standard3 / deviant3` for `walking_together`. This mapping was important because the ERP analysis depended on separating standard and deviant responses within each behavioral condition.

For the synchronization branch, the final condition groups were `natural`, `blocked`, and `sync`. Gait-event markers were simplified into heel-strike and toe-off events and used for stride extraction. Using the same event logic in both pipelines made the later comparison fairer, because differences between pipelines then came mainly from preprocessing and downstream analysis rather than from different label definitions.

4.2 Preprocessing: authors-inspired pipeline

The authors-inspired pipeline was built to follow the published analysis direction as closely as possible in our Python/MNE workflow [4]. It should be understood as a reconstruction, not as an exact copy of the original setup.

For each run, the raw EEG was loaded together with the main BIDS sidecar information. Channel types were read from `channels.tsv`, annotations were attached from the event files, and electrode positions were used when possible to build a usable montage.

The main preprocessing steps in this branch were:

- band-pass filtering from 1–120 Hz,
- no notch filtering,
- resampling to 250 Hz,
- bad-channel detection using a robust standard-deviation rule,
- optional RANSAC-based bad-channel detection when available,
- interpolation of detected bad channels,
- average re-referencing.

Artifact correction was then done with ICA. In this branch, the preferred method was extended Infomax ICA. The ICA was fitted on a 1 Hz high-pass filtered copy of the data, which made the decomposition more stable. Automatic component labeling was attempted with ICLabel when available [5], and components with brain probability below 0.70 were excluded.

One important point is that ASR-style cleaning was listed in the intended configuration, but it was not active in the final working Python implementation. So this step was not actually part of the final pipeline.

The cleaned continuous data were saved as FIF files together with quality-control outputs such as:

- before/after signal plots,
- PSD summaries,
- montage views,
- ICA component plots,
- and JSON summaries.

4.3 Preprocessing: our improved pipeline

Our own preprocessing pipeline used the same general structure, but with choices that were a bit more conservative and easier to inspect. It started from the same shared audit and event-mapping outputs, so later differences between pipelines mainly came from preprocessing itself.

The main preprocessing steps in this branch were:

- band-pass filtering from 0.1–40 Hz,
- notch filtering at 50 and 100 Hz,
- no resampling at the preprocessing stage,
- bad-channel detection using a robust standard-deviation rule,
- optional RANSAC-based bad-channel detection,
- interpolation of bad channels,
- average re-referencing.

ICA cleaning was then applied. In this branch, the preferred ICA method was Picard, with FastICA used as a fallback if Picard was not available. As in the authors-inspired branch, ICA was fitted on a 1 Hz high-pass filtered copy of the data. ICLabel was again used when available, but with a stricter threshold: components with brain probability below 0.80 were excluded.

This branch did not use ASR. The main goal here was to keep the preprocessing transparent, stable, and easier to rerun.

The outputs were stored in the same general format as the authors-inspired branch, including cleaned FIF files, ICA objects, PSD figures, montage plots, before/after raw plots, and run-level summaries.

4.4 Key differences between the two pipelines

Even though both pipelines shared the same audit, event mapping, and downstream analysis structure, the preprocessing settings were clearly different.

The main differences were:

- **Filtering:** the authors-inspired branch used 1–120 Hz, while our branch used 0.1–40 Hz with explicit notch filtering.
- **Resampling:** the authors-inspired branch resampled to 250 Hz, while our branch did not resample during preprocessing.
- **ICA method:** the authors-inspired branch used extended Infomax ICA, while our branch preferred Picard with FastICA as fallback.

- **ICLabel threshold:** the authors-inspired branch used a 0.70 brain-probability threshold, while our branch used 0.80.
- **ASR:** ASR was part of the intended authors-inspired configuration, but it was not active in the final implementation.

So, the two branches were similar enough for a fair comparison, but different enough to test whether preprocessing choices changed the final results.

4.5 Oddball ERP analysis

The oddball ERP analysis used the shared event mapping and the cleaned outputs from each preprocessing pipeline. The final behavioural conditions were standing, walking_alone, and walking_together, with trials further divided into standard and deviant stimuli.

For each cleaned run, annotations were converted into oddball event labels. EEG epochs were extracted from -0.2 s to 0.8 s around each event, with baseline correction from -0.2 s to 0 s. Epochs overlapping with marked bad annotations were rejected automatically, and an evoked response was retained only when at least five usable epochs were available for that label.

Averaging was performed at the run, subject-session, and group levels. The main contrast of interest was the deviant-minus-standard difference wave, computed separately for standing, walking alone, and walking together.

For summary measures, the ERP analysis focused on Cz and Pz. Peak metrics were extracted in the 0.25–0.5 s window, and topographic summaries were generated at 0.1 s, 0.2 s, 0.3 s, and 0.4 s. These settings were kept identical across both pipelines to ensure a fair comparison.

4.6 Walking synchronization time-frequency analysis

The walking synchronization branch used the cleaned EEG data together with the shared synchronization event mapping. The final conditions were natural, blocked, and sync. Unlike the oddball branch, this analysis focused on gait-locked rather than stimulus-locked activity.

Synchronization-related annotations were converted into a cleaner event table containing condition, role, and gait-event information. For stride construction, only participant gait events were used. A valid stride was defined as the interval between two consecutive heel strikes with one toe-off event in between. Only strides between 0.6 s and 2.0 s were retained.

After valid strides were identified, EEG segments were extracted and analysed using Morlet time-frequency methods. Each stride was then warped to a fixed cycle length of 200 points so that strides of different durations could be averaged on a common scale.

The two pipelines differed in several analysis choices. The authors-inspired pipeline used a 1.0 s pre-stride extension, a fixed toe-off alignment at 68% of the cycle, and log-ratio correction without a fixed pre-event baseline. Our pipeline used a 0.2 s pre-stride extension, a linear-cycle warp, and log-ratio correction with a baseline from -0.2 s to 0 s. Frequency settings also differed slightly: the authors-inspired branch analysed 3–40 Hz with 20 bins, while our branch analysed 3–35 Hz with 17 bins. In both cases, Morlet cycles increased from 4 to 10 across frequency.

For the final summaries, two predefined sensor-frequency ROIs were used: an alpha-mu ROI (7.5–12.5 Hz over C4, C6, CP4, and CP6) and a beta ROI (16–32 Hz over C1, Cz, and C2). For

each run and condition, these ROI summaries were converted into cycle curves and mean-power values, which were then pooled at the subject and group levels.

4.7 Decoding extension

The decoding analysis was added as an extension after the main oddball and synchronization branches were already working. Instead of averaging waveforms, this branch aimed to classify behavioural conditions directly from time-resolved EEG data.

The three pairwise comparisons were standing versus walking_alone, standing versus walking_together, and walking_alone versus walking_together. Epochs were extracted from -0.2 s to 0.8 s with baseline correction from -0.2 s to 0 s, then resampled to 100 Hz. Only subjects with enough usable epochs were included. The minimum number of epochs per condition was 40, and the maximum was capped at 250.

For each subject and condition pair, classes were balanced by random subsampling. A sliding estimator was trained across time using standardization followed by Linear Discriminant Analysis. Performance was measured with ROC AUC, where chance level is 0.5.

Cross-validation used stratified shuffle-split with five splits and a test size of 0.2. This produced one AUC time course per subject and condition pair, which was then averaged at the group level.

To simplify interpretation, three time windows were defined in advance: 0.0–0.2 s, 0.25–0.5 s, and 0.5–0.8 s. Mean AUC values within these windows were later used for summary tables and statistical testing.

4.8 Statistical comparison approach

The comparison strategy differed across the three analysis branches.

For the oddball ERP branch, the comparison was mainly descriptive and figure-based, focusing on grand-average waveforms, deviant-minus-standard difference waves, topographic comparisons, and peak summaries.

For the synchronization branch, statistical testing was more central. After ROI summaries were pooled to the subject level, paired condition comparisons were performed within each pipeline. The authors-inspired branch used Wilcoxon signed-rank tests, whereas our pipeline used paired t-tests. In both cases, p-values were corrected using false discovery rate correction, and effect sizes with bootstrap confidence intervals were also estimated.

For the decoding branch, the main question was whether window-level AUC values were above chance. For each condition pair and time window, a one-sample t-test against 0.5 was performed at the group level, followed by false discovery rate correction.

Overall, the statistical approach was kept simple. The main goal of the project was to build, reproduce, and compare complete EEG analysis pipelines, so the focus was on analyses that made the final outputs easier to interpret across pipelines.

Chapter 5

Results

5.1 Audit and event-mapping outcomes

The shared audit and event-mapping stage was completed successfully and gave both pipelines the same starting point. This was important because the later comparisons would not have been meaningful if the event definitions had differed from the beginning.

For the oddball branch, the final mapping separated the three behavioral conditions `standing`, `walking_alone`, and `walking_together`, and linked the correct standard and deviant labels to each one. For the synchronization branch, the final condition groups were `natural`, `blocked`, and `sync`, with gait markers parsed in a way that supported stride extraction. A summary of the final mapping used in the workflow is shown in Table 5.1.

Overall, this stage turned the dataset into a consistent analysis framework and removed one of the main sources of confusion from the earlier project stages

Table 5.1: Final event mapping used in the analysis workflow.

Raw event labels	Final condition
`standard1 / deviant1`	`standing`
`standard2 / deviant2`	`walking_alone`
`standard3 / deviant3`	`walking_together`
Synchronization labels	`natural`, `blocked`, `sync`
Gait markers	heel-strike, toe-off

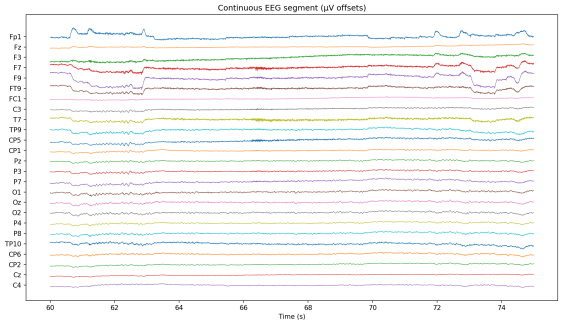
5.2 Preprocessing sanity checks and quality control

Both preprocessing branches produced cleaned outputs that could be used in all later stages of the project. To make the comparison fair, Figures 5.1 and 5.2 show representative outputs from the same subject and the same recording segment for both pipelines.

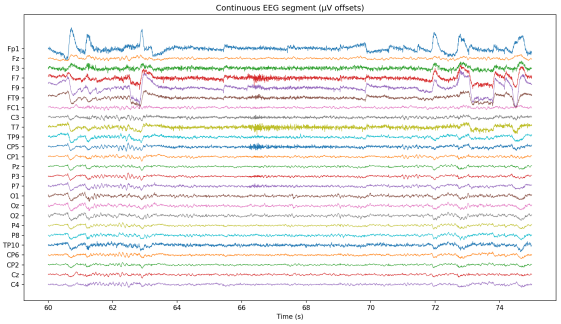
Figure 5.1 compares the continuous EEG before and after cleaning. In both pipelines, the cleaned signals are visibly more stable, with reduced high-frequency contamination and less irregular fluctuation in several frontal and temporal channels. At the same time, the overall signal structure is preserved, which suggests that preprocessing removed unwanted activity without making the data unusably flat.

Figure 5.2 shows the ICA decompositions from the same subject. The component maps are spatially structured in both pipelines, which indicates that the ICA step worked as intended. The exact component patterns are not identical, which is expected because the two pipelines use different filtering and ICA settings. Still, both decompositions produce components that are suitable for artifact inspection and removal.

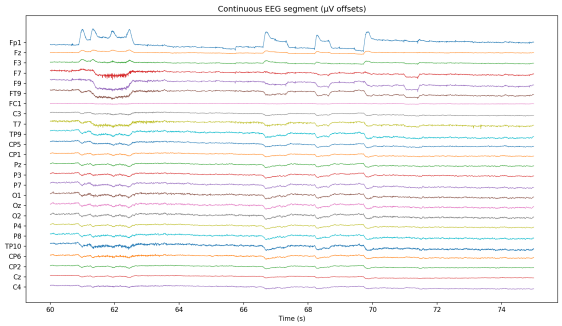
These figures do not replace a full QC appendix, but they are enough to show that both preprocessing branches reached a usable stage and produced data suitable for the later ERP, synchronization, and decoding analyses.



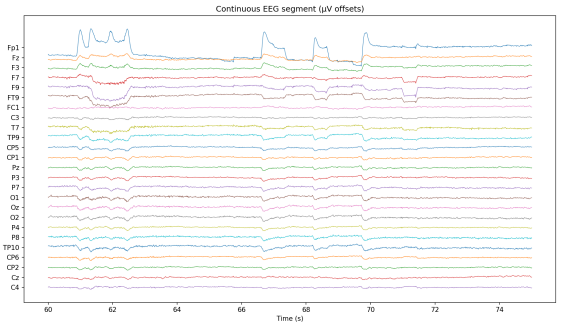
(a) Authors-inspired pipeline before cleaning



(b) Authors-inspired pipeline after cleaning

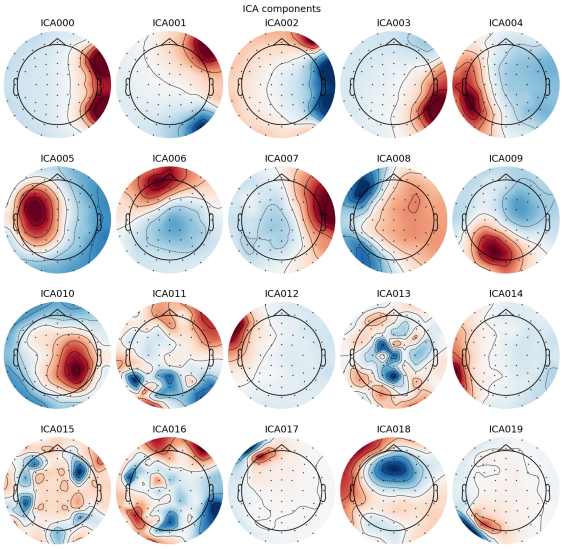


(c) Our pipeline before cleaning

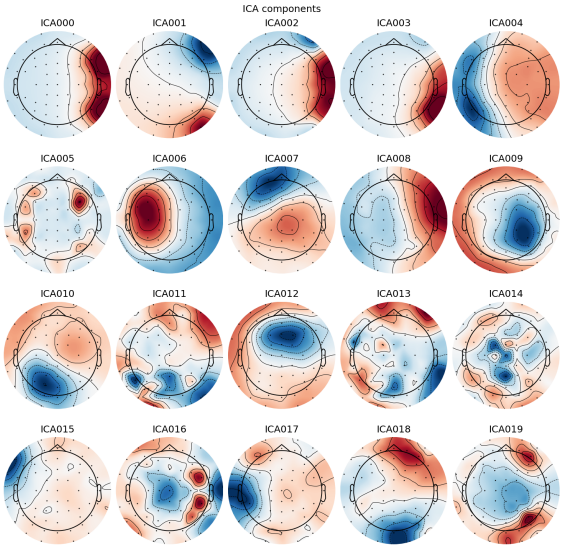


(d) Our pipeline after cleaning

Figure 5.1: Representative continuous EEG segment from the same subject, shown before and after preprocessing for the authors-inspired and improved pipelines. In both branches, cleaning reduces visible noise and unstable fluctuations while preserving the overall signal structure.



(a) Authors-inspired pipeline ICA decomposition



(b) Our pipeline ICA decomposition

Figure 5.2: Representative ICA component maps from the same subject for the two preprocessing pipelines.

5.3 Oddball ERP replication results

The oddball ERP branch ran successfully and produced the expected outputs across all three behavioral conditions: `standing`, `walking_alone`, and `walking_together`. In total, the analysis generated 216 run-level evoked cells, 216 subject-session cells, 18 group-average cells, 9 deviant-minus-standard difference waves, and 18 peak-metric rows.

Figure 5.3 shows the grand-average waveforms and difference waves for the `all_sessions` grouping. Across both Cz and Pz, the deviant responses become more positive than the standard responses in the later part of the epoch, especially around 0.3–0.4 s. This pattern is visible in all three behavioral conditions, although it is strongest in `standing` and somewhat weaker in `walking_alone`.

Overall, these results show that the final event mapping, epoching, and averaging steps worked as intended and that the oddball branch produced a clear group-level ERP effect.

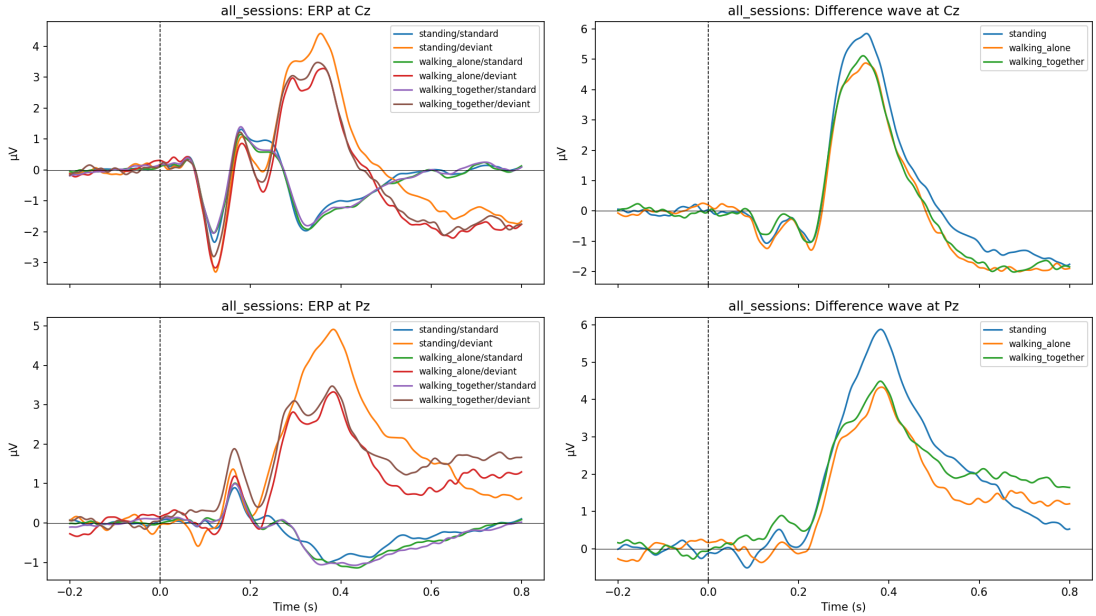


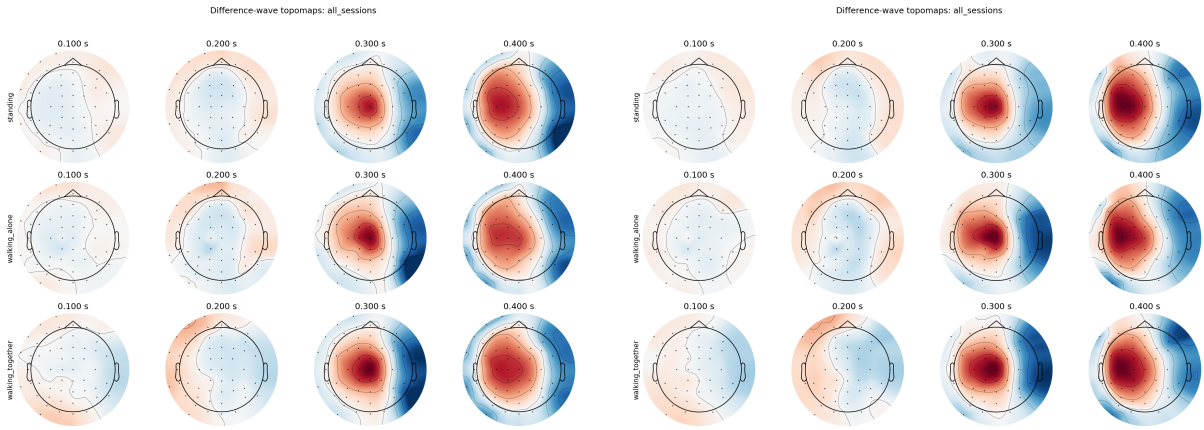
Figure 5.3: Grand-average oddball ERP waveforms and difference waves for the all_sessions grouping in the improved pipeline.

5.4 Authors vs. ours ERP comparison

The ERP comparison showed that both pipelines recovered a very similar difference-wave pattern at the group level. Figure 5.4 compares the all_sessions topographic summaries for the authors-inspired and improved pipelines across the main time points.

In both pipelines, the early maps at 0.1 s and 0.2 s are relatively weak, while the later maps at 0.3 s and 0.4 s show a clearer centro-parietal positivity with a corresponding negative field over the lateral right side. This overall pattern is consistent across standing, walking_alone, and walking_together, which suggests that both pipelines preserve the same main oddball effect.

At the same time, the improved pipeline produces slightly cleaner and more spatially defined maps, especially in the walking conditions. Even so, the overall interpretation stays the same in both branches. This supports the idea that the main difference between pipelines is in the fine detail of the output rather than in the basic result itself.



(a) Authors-inspired pipeline (b) Our pipeline

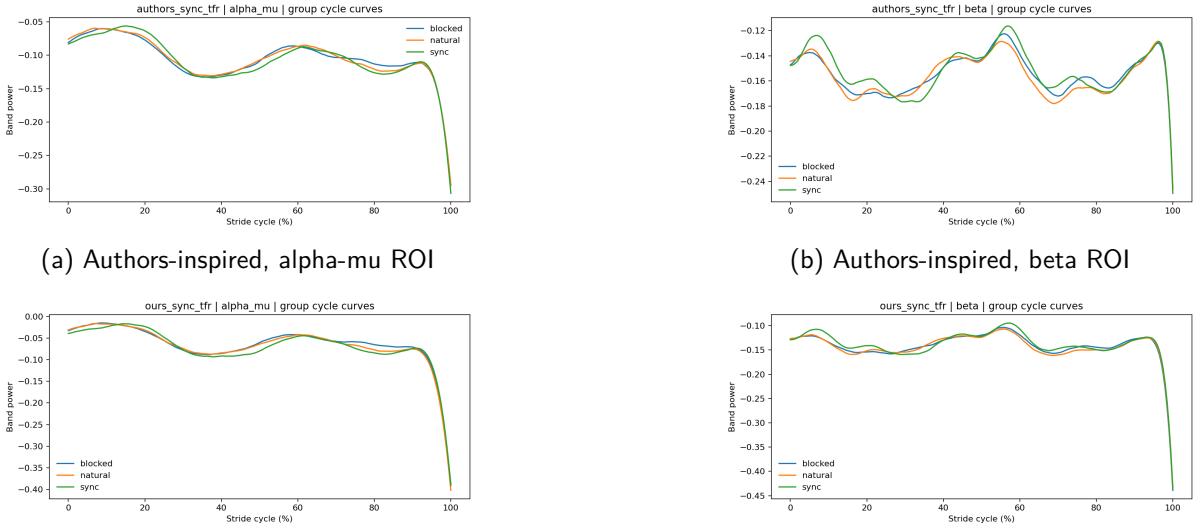
Figure 5.4: Difference-wave topomaps for all_sessions in the authors-inspired and improved pipelines.

5.5 Walking synchronization comparison

The synchronization comparison showed that both pipelines produced very similar group-level cycle patterns in the alpha-mu and beta ROIs. Figure 5.5 compares the group cycle curves for both pipelines and both ROIs.

Across the two pipelines, the overall shapes are broadly consistent. The main peaks and troughs appear at similar parts of the stride cycle, which suggests that both branches preserve the same general synchronization pattern. There are some amplitude differences, especially toward the end of the cycle, but these do not change the overall interpretation.

A sharp drop appears close to 100% of the stride cycle in all four plots. Since this pattern is present across both ROIs and both pipelines, it is more likely related to the cycle boundary or warping step than to a meaningful physiological effect. Overall, the comparison suggests that the two pipelines are similar enough to support the same synchronization analysis, while still showing some differences in scale and smoothness.



(a) Authors-inspired, alpha-mu ROI (b) Authors-inspired, beta ROI

(c) Our pipeline, alpha-mu ROI (d) Our pipeline, beta ROI

Figure 5.5: Group cycle curves for the alpha-mu and beta ROIs in the authors-inspired and improved pipelines.

Representative run-level TFR comparisons for the natural, blocked, and sync conditions are provided in Appendix B.2.

5.6 Decoding extension results

The decoding branch was completed as an extension based on the final oddball condition mapping. It covered the three pairwise comparisons `standing` versus `walking_alone`, `standing` versus `walking_together`, and `walking_alone` versus `walking_together`. In total, the analysis produced 108 epoch-summary rows, 5,454 subject-time rows, 162 subject-window rows, 303 group rows, and 9 statistical result rows.

At the group level, decoding performance stayed close to chance for all three comparisons. Small fluctuations above 0.5 were visible in some time ranges, especially for `standing` versus `walking_alone` and `standing` versus `walking_together`, but these effects were weak and not sustained across the epoch. The `walking_alone` versus `walking_together` comparison remained especially close to chance throughout.

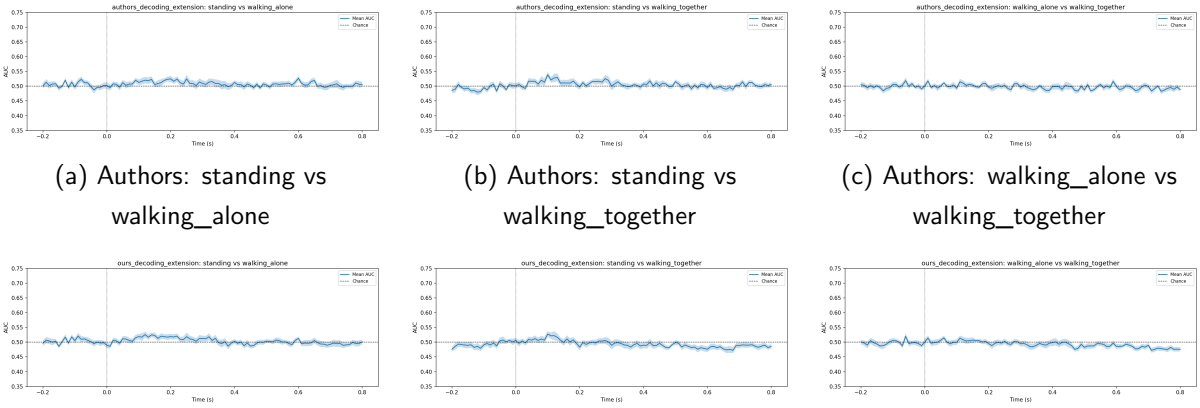
So, although the decoding branch was completed successfully from an implementation point of view, the result itself was weak. Its main value is that it shows the final project framework was flexible enough to support an additional analysis branch without major redesign.

5.7 Authors vs. ours decoding comparison

Figure 5.6 compares the group-level decoding AUC curves for the authors-inspired and improved pipelines across all three condition pairs. In both branches, the curves stay very close to chance for most of the epoch, which already suggests that the decoding result is weak.

The two pipelines show very similar overall behavior. For `standing` versus `walking_alone` and `standing` versus `walking_together`, both branches show only small increases above chance in the early to middle part of the epoch, but these are not strong or sustained enough to support a clear group-level decoding effect. For `walking_alone` versus `walking_together`, the curves remain almost flat around chance in both pipelines.

The comparison therefore does not suggest any meaningful decoding advantage for either pipeline. This matches the numerical summary, where the mean window-level AUC remained close to 0.5 in both cases. Overall, the decoding comparison supports a cautious interpretation of this branch as exploratory rather than as one of the main findings of the project.



(a) Authors: standing vs walking_alone

(b) Authors: standing vs walking_together

(c) Authors: walking_alone vs walking_together

(d) Ours: standing vs walking_alone

(e) Ours: standing vs walking_together

(f) Ours: walking_alone vs walking_together

Figure 5.6: Group-level decoding AUC curves for the authors-inspired and improved pipelines across the three pairwise condition comparisons. All curves remain close to chance level.

Chapter 6

Discussion

6.1 What was successfully reproduced

The main success of this project is that the full workflow was completed for both pipelines in a reproducible way. The shared audit and event mapping worked, both preprocessing branches ran successfully, and the oddball ERP, walking synchronization, and comparison stages were all completed. Decoding was also added and finished as an extension.

The walking synchronization branch is the closest match to the original study direction, while the oddball branch worked well as a complementary replication result. So, what was reproduced here was not just one figure, but the broader analysis logic of the dataset: shared event handling, two task branches, two preprocessing strategies, and direct comparison between them.

6.2 What changed when preprocessing changed

The two pipelines did affect the final outputs, but not enough to change the overall story. Both produced usable ERP results, usable synchronization summaries, and near-chance decoding results.

At the same time, the outputs were not identical. Differences in filtering, ICA setup, ICLabel threshold, and other implementation details changed some waveform, topography, and ROI details. This is expected in a real EEG project. Two reasonable pipelines can still support the same overall interpretation while differing in smaller details.

So the main point is not that one pipeline is the only correct one. It is that our improved pipeline made the workflow cleaner and more explicit, while the authors-inspired branch kept the comparison closer to the original study direction.

6.3 How the milestone problems were resolved

Compared with the earlier milestone stages, the final project is much more coherent. Early problems included unclear baseline choices, weak ERP structure, topography issues, and oddball conditions that were not yet mapped clearly enough.

These problems were improved mainly in three ways:

- the event mapping became much clearer, especially for standing, walking_alone, and walking_together,
- deviant-minus-standard difference waves made the ERP results easier to interpret,
- the project was reorganized so that the main replication branches were completed before decoding was added.

This made the final report much more focused and better aligned with the milestone feedback.

6.4 Limits of the authors-inspired reconstruction

The authors-inspired branch should not be described as an exact reimplementation of the original study environment. It was built in Python and MNE, so some parts had to be reconstructed rather than copied directly.

A clear example is ASR, which was part of the intended configuration but was not active in the final working implementation. Because of this, the branch is better described as authors-inspired rather than fully equivalent.

This is not a weakness by itself, but it does mean the report should stay honest about what was reproduced exactly and what was only approximated.

6.5 Why decoding should be treated as exploratory

The decoding branch worked technically, but it should still be treated as exploratory. The final AUC values stayed very close to chance for both pipelines, so this branch did not produce a strong discriminative result.

That does not make it useless. It still shows that the final framework was flexible enough to support an extra analysis branch. But the main conclusions of the project still come from the oddball ERP and walking synchronization results, not from decoding.

6.6 Limitations of the project

The project still has some clear limitations. The authors-inspired branch is a reconstruction, not an exact copy of the original setup. Some analysis choices were simplified, such as predefined ROIs and a simple decoding model. The final repository was also cleaned for submission, so it does not contain every intermediate output.

Even with these limits, the project achieved its main aim. It built a clear comparison framework, handled both main task branches in a reproducible way, and showed which findings were strong and which should be interpreted more carefully.

Chapter 7

Conclusion

7.1 Overall conclusion

This project aimed to build a reproducible EEG analysis framework around the OpenNeuro dataset ds004033 and compare an authors-inspired pipeline with our own improved pipeline. In the end, that goal was achieved. The final project included a shared audit and event-mapping workflow, two preprocessing branches, an oddball ERP branch, a walking synchronization time-frequency branch, and a decoding extension, all inside one consistent repository.

One of the main strengths of the project is that it does not rely on only one result. The same dataset supported two main analysis tracks: the oddball branch as a complementary ERP-based result, and the walking synchronization branch as the closer match to the original movement-focused study direction. Together, these made the project more complete.

The comparison between pipelines was also meaningful. The two pipelines were not identical, and some preprocessing choices did affect the final outputs, but the overall conclusions stayed stable. Both produced usable results, and the comparison figures made it possible to see both the similarities and the differences clearly. The decoding branch should be treated differently. It was implemented successfully and added value to the project framework, but the final AUC values stayed close to chance. Because of that, it should be seen as an exploratory extension rather than a main finding.

Overall, the project achieved its main aim. It turned a complex mobile EEG dataset into a reproducible workflow, completed the main replication-style analyses, and gave a fair comparison between an authors-inspired reconstruction and a cleaner improved pipeline.

7.2 Future work

There are several ways this project could be extended in the future.

- First, the authors-inspired branch could be pushed closer to the original study setup, especially in places where the current implementation had to approximate the original logic.
- Second, the walking synchronization branch could be developed further by testing additional ROIs, alternative stride-warping approaches, or stronger statistical analysis.
- Third, the oddball branch could be extended by looking more closely at participant-level variability, session differences, or more detailed topographic effects.

For decoding, future work should focus on improving the design rather than simply repeating the same model. More informative features or alternative classifiers may help, but they would need to be tested carefully to avoid overfitting.

Finally, the repository itself could be developed into a more reusable template for similar mobile EEG datasets. Since the shared audit, mapping, and modular workflow already worked well here, it could serve as a useful starting point for future work.

References

[1] J. E. M. Scanlon, N. S. Jacobsen, M. C. Maack, and S. Debener, "Outdoor walking: Mobile eeg dataset from walking during oddball task and walking synchronization task," *Data in Brief*, vol. 46, p. 108847, 2023.

[2] J. Scanlon, N. Jacobsen, M. Maack, and S. Debener, "Stepping in time: Alpha-mu and beta oscillations during a walking synchronization task," *NeuroImage*, vol. 253, p. 119099, 2022.

[3] A. Gramfort *et al.*, "Meg and eeg data analysis with mne-python," *Frontiers in Neuroscience*, vol. 7, p. 267, 2013.

[4] J. E. M. Scanlon, "jos_sync2: Code for the manuscript "stepping in time: beta and alpha-mu oscillations during walking synchronization task"," https://github.com/jscanlon275/jos_sync2, 2022, gitHub repository.

[5] L. Pion-Tonachini, K. Kreutz-Delgado, and S. Makeig, "Iclabel: An automated electroencephalographic independent component classifier, dataset, and website," *NeuroImage*, vol. 198, pp. 181–197, 2019.

Appendix A

Selected Configuration Tables

This appendix summarizes the main configuration settings used in the final workflow.

A.1 Preprocessing settings

Table A.1: Main preprocessing settings used in the two pipelines.

Setting	Authors pipeline	Our pipeline
Band-pass filter	1–120 Hz	0.1–40 Hz
Notch filter	none	50, 100 Hz
Resampling	250 Hz	none
Bad-channel detection	robust SD + RANSAC	robust SD + RANSAC
Interpolation	yes	yes
Reference	average	average
ICA method	extended Infomax	Picard (FastICA fallback)
ICA high-pass for fitting	1 Hz	1 Hz
ICLabel threshold	0.70	0.80
ASR	intended, not active in final run	no

A.2 Oddball ERP settings

Table A.2: Main oddball ERP settings used in both pipelines.

Setting	Value
Condition order	standing, walking_alone, walking_together
Stimulus order	standard, deviant
Epoch window	-0.2 to 0.8 s
Baseline window	-0.2 to 0.0 s
Reject by annotation	yes
Minimum epochs per label	5
Equalize within condition	no
ROI channels	Cz, Pz
Peak window	0.25 to 0.5 s
Topomap times	0.1, 0.2, 0.3, 0.4 s

A.3 Synchronization TFR settings

Table A.3: Main synchronization time-frequency settings in the two pipelines.

Setting	Authors pipeline	Our pipeline
Condition order	natural, blocked, sync	natural, blocked, sync
Pre-stride segment	1.0 s	0.2 s
Baseline window	no fixed window	-0.2 to 0.0 s
Baseline mode	log-ratio	log-ratio
Stride duration range	0.6–2.0 s	0.6–2.0 s
Cycle length	200 points	200 points
Warp mode	fixed toe percent	linear cycle
Toe-off alignment	68%	68% parameter retained
Frequency range	3–40 Hz	3–35 Hz
Number of frequencies	20	17
Morlet cycles	4 to 10	4 to 10
Alpha-mu ROI	C4, C6, CP4, CP6 / 7.5–12.5 Hz	same
Beta ROI	C1, Cz, C2 / 16–32 Hz	same
Stats method	Wilcoxon	paired t-test
FDR alpha	0.05	0.05

A.4 Decoding settings

Table A.4: Main decoding settings used in both pipelines.

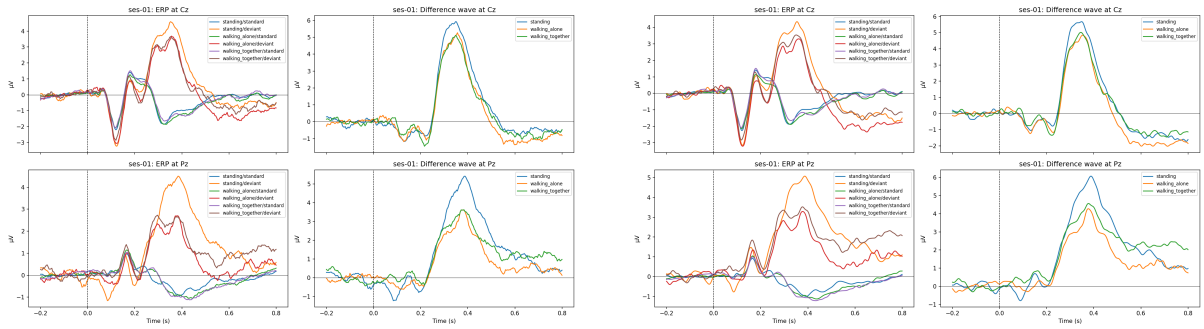
Setting	Value
Condition pairs	standing vs walking_alone; standing vs walking_together; walking_alone vs walking_together
Epoch window	-0.2 to 0.8 s
Baseline window	-0.2 to 0.0 s
Resampling for decoding	100 Hz
Minimum epochs per condition	40
Maximum epochs per condition	250
Cross-validation	stratified shuffle-split, 5 splits
Test size	0.2
Classifier	standardization + LDA
Metric	ROC AUC
Chance level	0.5
Windows used for summary	early (0.0–0.2 s), P3 (0.25–0.5 s), late (0.5–0.8 s)

Appendix B

Additional Figures

B.1 Additional oddball figures

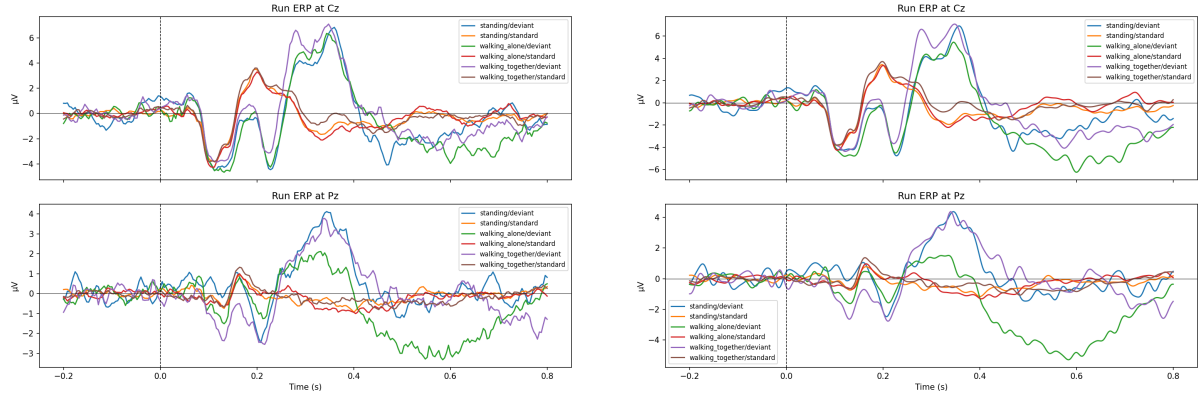
Additional oddball figures are included here to show selected subject-level and session-level results that support the main ERP findings reported in Chapter 5.



(a) Authors-inspired pipeline

(b) Our pipeline

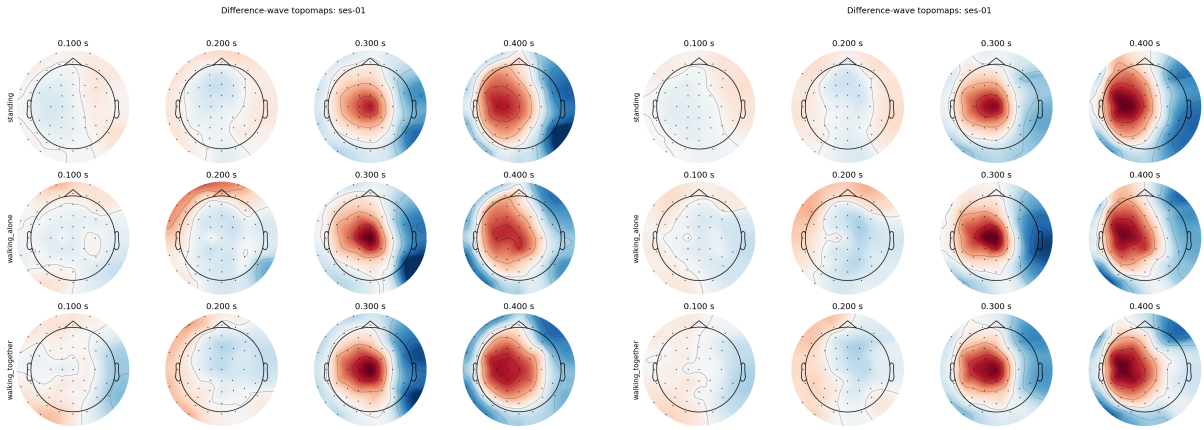
Figure B.1: Grand-average oddball ERP waveforms for ses-01.



(a) Authors-inspired pipeline

(b) Our pipeline

Figure B.2: Representative subject-level ROI ERPs at Cz and Pz.

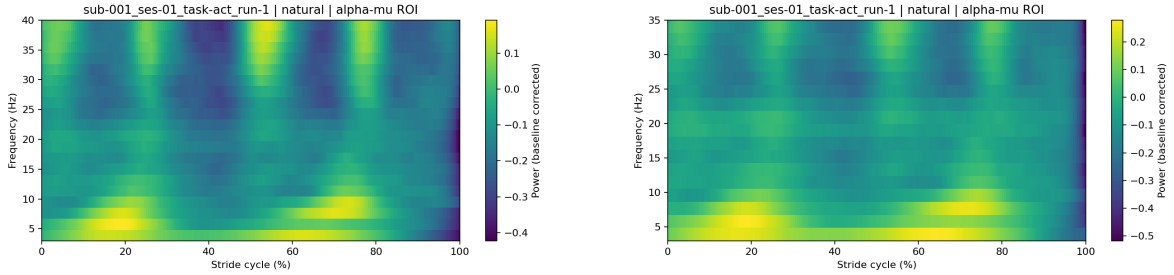


(a) Authors-inspired pipeline (b) Our pipeline

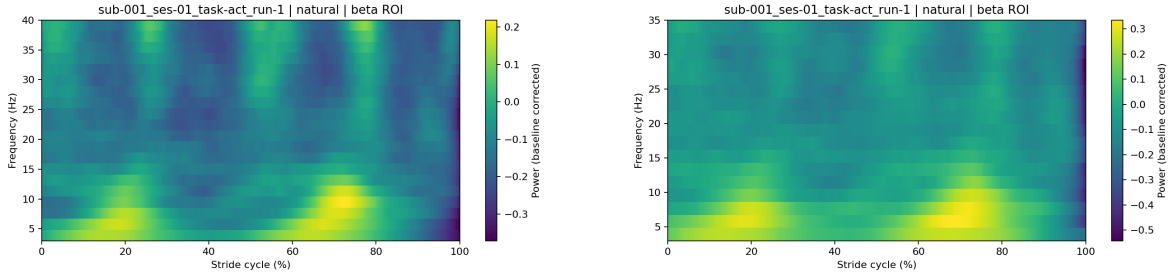
Figure B.3: Difference-wave topomaps for ses-01.

B.2 Additional synchronization comparison figures

Figures B.4, B.5, and B.6 show representative run-level TFR comparisons between the authors-inspired and improved pipelines for the natural, blocked, and sync conditions. In all three cases, the same subject and run are used to make the comparison direct.

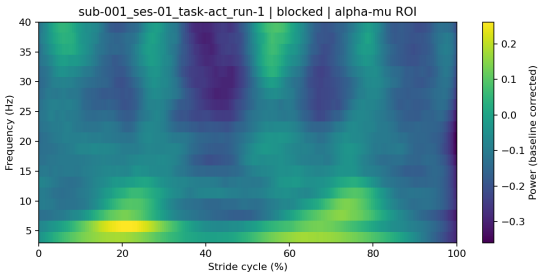


(a) Authors-inspired, alpha-mu ROI (b) Our pipeline, alpha-mu ROI

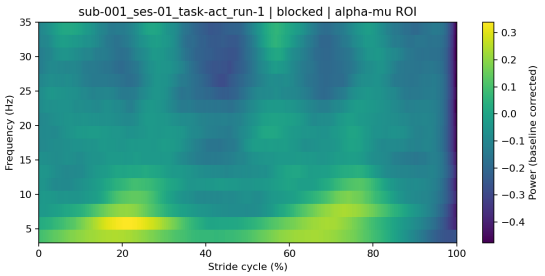


(c) Authors-inspired, beta ROI (d) Our pipeline, beta ROI

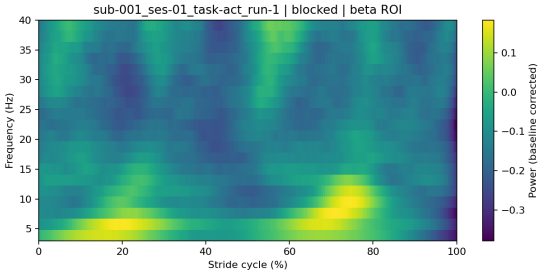
Figure B.4: Representative run-level TFR comparison for the natural condition.



(a) Authors-inspired, alpha-mu ROI



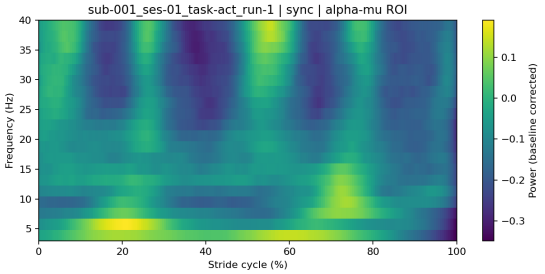
(b) Our pipeline, alpha-mu ROI



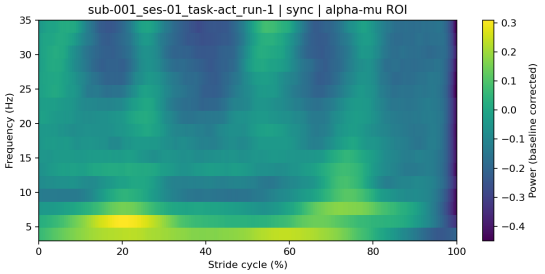
(c) Authors-inspired, beta ROI

(d) Our pipeline, beta ROI

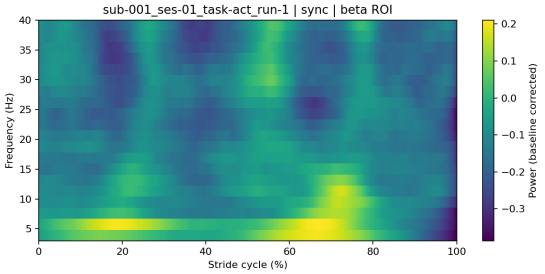
Figure B.5: Representative run-level TFR comparison for the blocked condition.



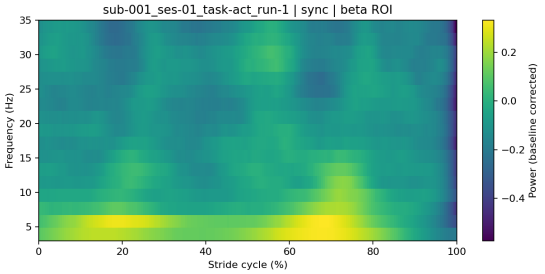
(a) Authors-inspired, alpha-mu ROI



(b) Our pipeline, alpha-mu ROI



(c) Authors-inspired, beta ROI



(d) Our pipeline, beta ROI

Figure B.6: Representative run-level TFR comparison for the sync condition.

Appendix C

Reproducibility Notes and Run Order

This appendix gives a short summary of how the final repository is organized and how the analysis workflow can be reproduced. The lightweight project repository was cleaned for submission, so it does not include the full raw dataset or every large intermediate output. Instead, it keeps the code, notebooks, configuration files, and selected figures needed to understand and rerun the workflow.

C.1 Repository link

The final cleaned project repository used for this report is available at https://github.com/KishoreKhan0/KishoreKhan_WS2025_EEGSemesterProject

C.2 Dataset location

The project is based on the OpenNeuro dataset `ds004033`. The dataset itself is not included in the lightweight repository and must be downloaded separately. In the final project structure, the expected location is: `data/ds004033/`

The repository also includes a short `README` inside the `data/` folder to explain this expected layout.

C.3 Environment setup

The repository includes both `environment.yml` and `requirements.txt` so that the software environment can be recreated. In practice, the preferred option is to build the environment from `environment.yml`, since this reflects the final project setup more closely. The `requirements.txt` file can also be used if a lighter installation route is preferred.

C.4 Main repository structure

The final repository is organized into a small number of main folders:

- `shared/`: common notebooks, scripts, and reusable source code
- `authors_pipeline/`: outputs and notebook structure for the authors-inspired branch
- `ours_pipeline/`: outputs and notebook structure for the improved branch
- `configs/`: YAML configuration files for the main analysis stages
- `results/`: selected lightweight figures kept for reporting
- `data/`: expected dataset location

This structure was chosen so that both pipelines could share the same early workflow while still keeping their outputs separate.

C.5 Notebook run order

The shared notebooks in the repository follow the project workflow in order. A full run of the analysis can be understood through the following sequence:

1. `00_data_audit.ipynb`
2. `01_event_mapping.ipynb`
3. `02_preprocessing.ipynb`
4. `03_oddball_replication.ipynb`
5. `04_pipeline_comparison.ipynb`
6. `05_sync_tfr_analysis.ipynb`
7. `06_sync_comparison.ipynb`
8. `07_decoding_analysis.ipynb`
9. `08_decoding_comparison.ipynb`

This notebook sequence reflects the same logic used in the report: first the shared setup stages, then the main analysis branches, and finally the comparison stages.

C.6 Script entry points

In addition to the notebooks, the repository also includes script-based entry points for rerunning the workflow more directly. The main scripts are:

- `run_data_audit.py`
- `run_event_mapping.py`
- `run_preprocessing.py`
- `run_oddball_replication.py`
- `run_sync_tfr_analysis.py`
- `run_sync_comparison.py`
- `run_decoding_analysis.py`
- `run_decoding_comparison.py`

C.7 Configuration files

The main settings of the workflow are stored in the `configs/` folder. This includes configuration files for preprocessing, oddball analysis, synchronization analysis, decoding, event mapping, and pipeline comparison. Keeping these settings in YAML files was helpful because it reduced hidden parameter changes inside notebooks and made the final workflow easier to inspect.

C.8 Final note

The final repository was designed to be lightweight, readable, and reproducible. It is not a complete archive of every raw and intermediate file produced during development, but it does preserve the main workflow, final configuration, and selected outputs needed to understand and rerun the project.